

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. Шухова»
(БГТУ им. В. Г. Шухова)



Кафедра программного обеспечения вычислительной
техники и автоматизированных систем

Лабораторная работа № 10

по дисциплине: «Основы программирования»

на тему: «Подготовка прикладной библиотеки на языке C»

Вариант 4: Частотный анализ текста

Выполнил: студент группы ВТ-252
ГУЙЛАЙ Мохаммед Эйса Мохаммед

Проверили:
доц. Островский Алексей Мичеславович,
асс. Стеценко Оксана Николаевна

Белгород, 2026

Цель работы

Получить практические навыки проектирования, реализации, сборки, документирования и тестирования прикладной библиотеки на языке C: научиться выделять публичный интерфейс, скрывать детали реализации, собирать разделяемую версию `.so`, использовать её из Python через `ctypes`, вести историю разработки в git, применять Makefile, статические анализаторы и санитайзеры, а также подготавливать автоматическую документацию средствами Doxygen.

Вариант 4 — Частотный анализ текста

Разработать библиотеку **freqlib** для частотного анализа ASCII-текста. Предусмотрены следующие задачи:

1. подсчёт частот символов ASCII;
 2. подсчёт частот слов ограниченной длины;
 3. поиск *top-k* наиболее частых слов.
-

1. Структура проекта

```
freqlib/  
  include/  
    freqlib.h           public API with Doxygen comments  
  src/  
    freqlib.c           library implementation  
    main.c             demo application  
  tests/  
    test_freqlib.c      C unit tests (43 cases)  
    test_freqlib.py     Python ctypes tests (22 cases)  
  docs/  
    Doxyfile            Doxygen configuration  
  reports/             static-analysis output (.gitignore)  
  build/               compiled artifacts (.gitignore)  
  Makefile  
  README.md  
  .gitignore
```

2. Публичный интерфейс библиотеки

2.1 Коды ошибок

Константа	Значение
FREQ_OK (0)	Операция выполнена успешно
FREQ_ERR_NULL (-1)	Передан NULL-указатель
FREQ_ERR_EMPTY (-2)	Пустая входная строка или нет слов
FREQ_ERR_ALLOC (-3)	Ошибка выделения памяти
FREQ_ERR_PARAM (-4)	Некорректное числовое значение

2.2 Типы данных

```
1  /* Character frequency table (stack-allocated, no heap) */
2  typedef struct {
3      size_t counts[128]; /* counts[c] == occurrences of ASCII
4      char c */
5      size_t total;      /* total ASCII characters seen
6      */
7  } char_freq_t;
8
9  /* One word + its occurrence count */
10 typedef struct {
11     char    word[64];    /* NUL-terminated, max 63 bytes
12     */
13     size_t count;
14 } word_entry_t;
15
16 /* Dynamic word-frequency table (heap-allocated entries) */
17 typedef struct {
18     word_entry_t *entries;
19     size_t        size;    /* distinct words stored */
20     size_t        cap;    /* allocated capacity */
21 } word_freq_t;
```

2.3 Функции

```
1  int  freq_char_count(const char *text, char_freq_t *out);
2  void freq_char_free (char_freq_t *cf);
3
4  int  freq_word_count(const char *text, word_freq_t *out);
5  void freq_word_free (word_freq_t *wf);
6
7  int  freq_top_k(const word_freq_t *wf, size_t k,
8                  word_entry_t **result, size_t *result_n);
```

2.4 Правила владения памятью

- `freq_char_count` заполняет структуру на стеке вызывающего; освобождение — `freq_char_free` (обнуляет поля).
 - `freq_word_count` выделяет `wf->entries` через `malloc/realloc`; освобождение — `freq_word_free`.
 - `freq_top_k` выделяет новый массив результата; вызывающая сторона освобождает его через `free()`.
-

3. Описание алгоритмов

Подсчёт символов. За один проход строки инкрементируется `counts[c]` для каждого символа с `co` значением меньше 128. Сложность: $O(n)$, где n — длина строки.

Подсчёт слов. Два указателя: пропуск не-алфавитно-цифровых символов, затем сбор «слова» (до 63 байт). Слово ищется в динамическом массиве линейным поиском; при совпадении счётчик инкрементируется, при отсутствии добавляется новая запись (ёмкость удваивается начиная с 16). Сложность: $O(m \cdot n)$ в худшем случае, где m — число слов, n — число различных слов.

Top-k. Создаётся копия массива `entries`, сортируется по убыванию счётчика через `qsort` (при равенстве — по алфавиту), возвращаются первые $\min(k, |wf|)$ элементов. Сложность: $O(n \log n)$.

4. Фрагменты ключевых файлов

4.1 Заголовочный файл `include/freqlib.h` (фрагмент)

```
1 #ifndef FREQLIB_H
2 #define FREQLIB_H
3
4 #include <stddef.h>
5
6 #define FREQ_OK          0
7 #define FREQ_ERR_NULL   -1
8 #define FREQ_ERR_EMPTY  -2
9 #define FREQ_ERR_ALLOC  -3
10 #define FREQ_ERR_PARAM  -4
11
```

```

12 #define  FREQ_ASCII_SIZE  128
13 #define  FREQ_WORD_MAXLEN  63
14
15 typedef struct {
16     size_t counts[128];
17     size_t total;
18 } char_freq_t;
19
20 int  freq_char_count(const char *text, char_freq_t *out);
21 void freq_char_free (char_freq_t *cf);
22
23 /* word_freq_t, word_entry_t, freq_word_count,
24    freq_word_free, freq_top_k ... */
25 #endif

```

4.2 Реализация freq_word_count (ключевой фрагмент)

```

1  const char *p = text;
2  while (*p != '\0') {
3      while (*p != '\0' && !isalnum((unsigned char)*p)) p++;
4      if (*p == '\0') break;
5
6      char buf[FREQ_WORD_MAXLEN + 1];
7      size_t len = 0;
8      while (*p != '\0' && isalnum((unsigned char)*p)) {
9          if (len < FREQ_WORD_MAXLEN) buf[len++] = *p;
10         p++;
11     }
12     buf[len] = '\0';
13
14     int idx = wf_find(out, buf);
15     if (idx >= 0) {
16         out->entries[idx].count++;
17     } else {
18         if (out->size == out->cap) wf_grow(out);
19         memcpy(out->entries[out->size].word, buf, len + 1);
20         out->entries[out->size].count = 1;
21         out->size++;
22     }
23 }

```

4.3 Makefile (фрагмент)

```

CC      := gcc
CFLAGS  := -Wall -Wextra -Wpedantic -std=c11 -Iinclude

shared:
    $(CC) $(CFLAGS) -O2 -fPIC -shared -o build/libfreqlib.so
    src/freqlib.c

```

```

sanitize:
    $(CC) $(CFLAGS) -g -fsanitize=address,undefined \
        -o build/test_san tests/test_freqlib.c src/freqlib.c
        -Iinclude
    build/test_san

```

4.4 Python-вызов через ctypes (фрагмент)

```

1 import ctypes
2 lib = ctypes.CDLL("build/libfreqlib.so")
3
4 class CharFreq(ctypes.Structure):
5     _fields_ = [("counts", ctypes.c_size_t * 128),
6                 ("total", ctypes.c_size_t)]
7
8 lib.freq_char_count.argtypes = [ctypes.c_char_p,
9                                  ctypes.POINTER(CharFreq)]
10 lib.freq_char_count.restype = ctypes.c_int
11
12 cf = CharFreq()
13 rc = lib.freq_char_count(b"hello world", ctypes.byref(cf))
14 assert rc == 0
15 assert cf.counts[ord('l')] == 3
16 lib.freq_char_free(ctypes.byref(cf))

```

5. Команды сборки и результаты

```

$ make all
[shared] build/libfreqlib.so
[app] build/app
43 passed, 0 failed
Build complete.

$ make run
freqlib demo
Character frequencies (printable ASCII, count > 0):
    ' '      67
    'e'      31    't'      28    'o'      27    's'      23
    'a'      22    ...
    total: 331
Distinct words: 48
Top 10 most frequent words:
    1. to 7
    2. a 3
    3. and 3

```

```

4.   the           3
5.   The           2
...
Done.

$ make syntax
[syntax] OK - no errors

$ make sanitize
43 passed, 0 failed    (AddressSanitizer + UBSan: clean)

$ make py-test
=== freqlib Python test suite ===
22 passed, 0 failed

$ make analyze
--- gcc -fanalyzer ---    (no warnings)
--- cppcheck ---          (no errors)

$ make docs-html
[docs]      docs/html/index.html

```

6. Таблица тестовых случаев

Входные данные	Функция	Ожидаемый результат	Итог
NULL вместо текста	freq_char_count	FREQ_ERR_NULL	PASS
NULL вместо out	freq_char_count	FREQ_ERR_NULL	PASS
Пустая строка ""	freq_char_count	FREQ_ERR_EMPTY	PASS
"aabbcc"	freq_char_count	counts: a=2, b=2, c=1	PASS
Один символ "z"	freq_char_count	counts[z]=1, total=1	PASS
Три пробела " "	freq_char_count	counts[' ']=3, total=3	PASS
После freq_char_free		total = 0	PASS
NULL вместо текста	freq_word_count	FREQ_ERR_NULL	PASS
Пустая строка	freq_word_count	FREQ_ERR_EMPTY	PASS
Только знаки "!!! ???"	freq_word_count	FREQ_ERR_EMPTY	PASS
"the cat and the dog"	freq_word_count	size=4, the:2	PASS
Одно слово "hello"	freq_word_count	size=1, count=1	PASS
Знаки препинания	freq_word_count	hello:2	PASS

Слово 100 символов	<code>freq_word_count</code>	обрезается до 63, OK	PASS
Три регистра одного слова	<code>freq_word_count</code>	3 разные записи	PASS
<code>k=0</code>	<code>freq_top_k</code>	<code>FREQ_ERR_PARAM</code>	PASS
NULL аргумент	<code>freq_top_k</code>	<code>FREQ_ERR_NULL</code>	PASS
"a b a c a b", <code>k=2</code>	<code>freq_top_k</code>	[a:3, b:2]	PASS
$k >$ числа слов	<code>freq_top_k</code>	все слова	PASS
$k = 1$	<code>freq_top_k</code>	только «a»	PASS
Пустая таблица	<code>freq_top_k</code>	<code>resn=0</code> , <code>res=NULL</code>	PASS

Итого: **43** C-теста, **22** Python-теста — все пройдены

7. История разработки (git log -oneline)

```
8d2e082 docs: add changelog; note fix for no-word input edge
case
09b7b0c test(python): add ctypes integration tests with
explicit argtypes/restype
5c01477 test(c): add unit tests covering typical, boundary and
error cases
be225f6 build: add demo application and Makefile with all
targets
d504960 feat(src): implement freq_char_count, freq_word_count
and freq_top_k
e2aa5ae feat(include): add public header with types, error
codes and Doxygen comments
67cc512 init: scaffold project structure, .gitignore and README
```

8. Выводы

В ходе выполнения лабораторной работы №10 реализована библиотека **freqlib** для частотного анализа ASCII-текста (вариант 4).

Реализовано:

- Публичный интерфейс в `include/freqlib.h` с полными Doxygen-комментариями и единой схемой кодов ошибок.
- Три функциональных группы API: `freq_char_count`, `freq_word_count`, `freq_top_k`.

- Разделяемая библиотека `libfreqlib.so` (собирается с `-fPIC -shared`).
- Демонстрационное приложение `src/main.c`.
- **43** C-теста и **22** Python-теста через `ctypes` — все пройдены.
- Makefile с 12 целями: `all`, `shared`, `app`, `run`, `test`, `py-test`, `syntax`, `analyze`, `sanitize`, `docs-html`, `docs-pdf`, `clean`.
- История разработки в git: 7 осмысленных коммитов.

Найденные ошибки и исправления:

- Изначально `freq_word_count` возвращала `FREQ_OK` для строк, не содержащих слов (например, `"!!! ???"`); исправлено — теперь возвращается `FREQ_ERR_1`.
- Статический анализ (`cppcheck`, `gcc -fanalyzer`) и санитайзеры (`ASan + UBSan`) ошибок не выявили.

Ограничения и возможные расширения:

- Поиск слова линейный $O(n)$ — при больших текстах можно заменить хеш-таблицей.
- Сравнение слов чувствительно к регистру — можно добавить нечувствительный режим.
- Максимальная длина слова фиксирована (63 байта) — можно сделать настраиваемой через параметр функции.